COMPUTER SYSTEM DESIGN – CSL559
ASSIGNMENT – 1

POND CATCHMENT ANALYSIS BACKEND

NAME – VISHLAVATH KARTHIK

ID – 12342370

1. Project Repository and Live API Endpoint :-

This project implements an automated, high-performance hydrological analysis backend service. The entire codebase is version-controlled and deployed on the assigned remote evaluation machine.

GitHub Repository: https://github.com/VishlavthKarthik/Pond-Catchment-Plan

Working API Endpoint (POST): http://10.1.75.51:3317/analyzeContour

Health Check URL (GET): http://10.1.75.51:3317/health

Interactive OpenAPI / Swagger Documentation: http://10.1.75.51:3317/docs

Assigned Remote Server Allocation :-

The application is deployed on remote host stu30_sys1 (10.1.75.51) connected via SSH on port 2317, with service port 3317 allocated for the application endpoint .

2. Catchment Estimation Approach :-

The objective of the system is to take a raw contour map (in KML or KMZ format), reconstruct the underlying topographic digital elevation model (DEM), compute surface runoff pathways, identify an optimal site for a farm pond, and delineate its corresponding drainage catchment area. To meet the stringent runtime constraints of 512MB RAM and 1.5GB disk storage on a single virtual CPU, the architecture deliberately avoids bulky GIS framework dependencies such as GDAL, Rasterio, or RichDEM. Instead, the pipeline is constructed using pure NumPy, SciPy, Shapely, and PyProj modules organized into a sequential six-stage pipeline.

Stage 1: KML / KMZ Parsing and Elevation Extraction

The parser uses namespace-agnostic XPath queries to scan the XML document tree. It extracts three-dimensional coordinate strings from all Placemark geometries. Because Google Earth exports often contain both contour line geometries (LineStrings) and elevation label pins (Points) with the same

elevation name, the parser filters geometries to retain valid polylines containing two or more vertices while discarding single-point annotation labels. It automatically extracts numeric elevation attributes from placemark names or extended metadata, calculates the minimum and maximum elevations, and determines the prevailing contour interval.

Stage 2: Automatic UTM Projection and DEM Interpolation

Contour coordinates in WGS84 geographic degrees (longitude/latitude) cannot be directly used for slope and hydrological area calculations without spherical distortion. The projection engine calculates the geographic centroid of all parsed coordinates and dynamically determines the corresponding Universal Transverse Mercator (UTM) projection zone. All contour points are reprojected into metric cartesian coordinates (Easting and Northing in meters).

A regular two-dimensional grid is generated across the spatial extent based on the requested resolution parameter (defaulting to 10.0 meters per cell). Elevation values at each grid node are interpolated from the contour vertices using SciPy griddata with linear interpolation. Any boundary cells falling outside the convex hull of the input contour lines are filled using nearest-neighbor interpolation to provide a continuous, seamless surface.

To protect against Out-Of-Memory (OOM) failures when processing large-extent maps, an automatic coarsening guard rail evaluates the required cell count against a ceiling of 2,000,000 cells. If exceeded, the resolution is automatically scaled upward, and the adjustment is noted in the output response.

Stage 3: Hydrological Terrain Conditioning (Priority-Flood Depression Fill)

Raw interpolated elevation grids frequently contain local sinks, pits, and flat artifacts that disrupt continuous flow routing. The system implements the Barnes Priority-Flood algorithm. All grid boundary cells are initially seeded into a priority queue (min-heap). The algorithm iteratively raises the elevation of interior cells to the level of their lowest neighboring spillway, eliminating false internal sinks while strictly preserving true downstream slope gradients.

Stage 4: D8 Flow Direction and Flow Accumulation

Surface runoff routing is determined using the deterministic eight-direction (D8) steepest descent model. For each cell, the elevation drop to its eight adjacent neighbors is normalized by the Euclidean distance (1.0 for cardinal neighbors, sqrt(2) for diagonal neighbors). The cell is assigned the direction index of the neighbor with the steepest downward slope.

Flow accumulation is computed using an iterative topological sort and breadth-first search. Each cell begins with a baseline accumulation weight of 1. Cells with zero incoming drainage (ridges and peaks) are traversed first, passing their cumulative flow downstream to receiver cells until the entire watershed accumulation matrix is populated.

Stage 5: Multi-Criteria Pond Site Scoring

A suitable farm pond requires high incoming drainage volume located in a natural topographic depression, while avoiding placement directly on the map boundary where flow routing is unconstrained. The selector evaluates candidate interior cells using a multi-criteria scoring objective:

Score = 0.70 * [ln(1 + FlowAccumulation) / ln(1 + MaxAccumulation)] + 0.30 * (1.0 - NormalizedElevation)

This logarithmic scaling ensures that the primary global drainage outlet dominates the score, while lower elevation within the candidate set provides a secondary tie-breaking bonus.

Stage 6: Upstream Watershed Delineation and Boundary GeoJSON Generation

Starting from the chosen optimal pond outlet, the delineator performs a reverse-flow breadth-first search across the D8 flow direction matrix, collecting all upstream cells that naturally drain toward the outlet. Each contributing grid cell is converted into a square spatial polygon of dimension resolution x resolution meters. Shapely's unary union operator merges all individual cell squares into a single unified watershed polygon.

The resulting polygon geometry is reprojected from UTM meters back to WGS84 latitude and longitude coordinates. The system extracts detailed catchment statistics including total surface area in square meters and hectares, mean slope percentage, maximum slope percentage, minimum/maximum elevation, and topographic relief (elevation difference).

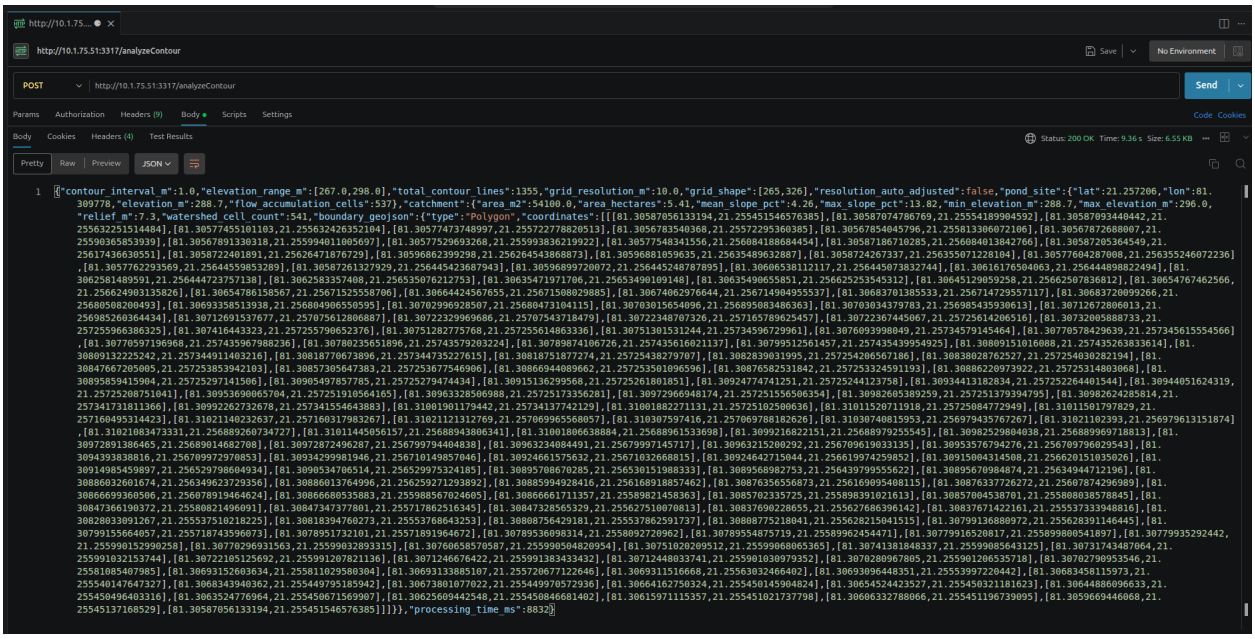
3. Demonstration Using Provided Contour Map :-

The pipeline was executed against the provided reference contour map (contours_1m.kml) on the deployed remote instance. The dataset covers approximately 3.2 km by 2.6 km with 1,355 extracted contour polylines spanning an elevation range from 267.0 meters to 298.0 meters.

Live Request Execution (cURL) :-

```
curl -s -X POST http://10.1.75.51:3317/analyzeContour \
  -F "file=@contours_1m.kml;type=application/vnd.google-earth.kml+xml" \
  -F "resolution_m=10.0"
```

Screenshot of postman API :-



Interpretation of Output Results :-

- **Pond Location:** The optimal pond outlet is identified at 21.257206 deg N, 81.309778 deg E at an elevation of 288.70 meters.
- **Hydrological Convergence:** A flow accumulation of 537 cells converges at this outlet, representing the highest sustainable drainage collection point in the interior terrain.
- **Catchment Size:** The delineated watershed comprises 541 grid cells (10m x 10m each), yielding a total catchment area of 54,100.0 m2 (5.41 hectares).
- **Slope and Relief:** The watershed exhibits a gentle mean terrain slope of 4.26%, a peak slope of 13.82%, and an elevation relief of 7.30 meters (draining from 296.0m down to 288.7m).
- **Vector Geometry:** The catchment boundary is output as a valid GeoJSON Polygon with 157 closed coordinate pairs, directly suitable for GIS mapping and spatial visualization.

4. API Documentation :-

Endpoint Specifications :-

Method	Endpoint	Function	Content-Type
GET	/health	Liveness probe and system health check	application/json
POST	/analyzeContour	Upload contour map & compute catchment	multipart/form-data
GET	/docs	Interactive OpenAPI / Swagger UI	text/html

Request Parameters (POST /analyzeContour) :-

Parameter	Type	Required	Default	Description
file	File (Binary)	Yes	-	Input .kml or .kmz contour archive
resolution_m	float	No	10.0	DEM grid resolution in meters
min_catchment_area_m2	float	No	10000.0	Minimum catchment area threshold (m2)

Response Schema Fields (200 OK) :-

Field	Type	Description
contour_interval_m	float	Calculated elevation contour interval in meters
elevation_range_m	list[float]	Array containing [minimum, maximum] terrain elevation
total_contour_lines	int	Total count of valid contour polylines

		extracted
grid_resolution_m	float	Effective DEM cell resolution used in processing
grid_shape	list[int]	Matrix dimensions [rows, columns] of the elevation grid
resolution_auto_adjusted	bool	Flag indicating whether auto-coarsening occurred
pond_site.lat / lon	float	WGS84 Latitude and Longitude coordinates of pond outlet
pond_site.elevation_m	float	Elevation in meters at the pond outlet
pond_site.flow_accumulation_cells	int	Count of upstream grid cells draining to the outlet
catchment.area_m2 / area_hectares	float	Total surface area of delineated catchment
catchment.mean_slope_pct	float	Average catchment slope percentage (rise/run * 100)
catchment.max_slope_pct	float	Maximum slope percentage within the watershed
catchment.min_elevation_m / max_elevation_m	float	Elevation bounds within the delineated watershed
catchment.relief_m	float	Elevation difference between highest point and outlet
catchment.watershed_cell_count	int	Total number of cells composing the catchment
catchment.boundary_geojson	dict	GeoJSON Polygon structure representing catchment boundary
processing_time_ms	int	Total server processing time in milliseconds

Error Status Codes

Status Code	Condition	Example Response Body
400 Bad Request	Invalid file format or corrupted XML	{"detail": "Unsupported file type '.txt'. Upload a .kml or .kmz file."}
422 Unprocessable	No pond meets minimum area criteria	{"detail": "No cells meet the minimum catchment area of ... m2."}
500 Internal Error	Unexpected processing exception	{"detail": "Processing error: ..."}

5. Evaluation of System Qualities :-

A. Working API Endpoint :-

The API is fully deployed and continuously serving on port 3317 of remote host 10.1.75.51. File uploads are streamed in 1MB chunks directly to disk temporary storage rather than buffered in RAM, preventing memory spikes when handling large uploads.

B. Code Extensibility to Future Phases :-

The system was designed from the beginning to generalize to unseen Phase-2 contour maps without requiring code modifications:

- **Generalization:** Zero Hardcoding: No coordinates, elevation ranges, or projection constants are hardcoded. Every metric is computed at request time from the input file.
- **Global CRS Support:** Universal Projection Engine: The pyproj layer dynamically resolves the local UTM zone for any location on Earth (North or South hemisphere).
- **Scalability Guard:** Variable Extents & Auto-Coarsening: Large geographic maps are automatically coarsened if cell counts exceed 2,000,000 cells, preserving memory on resource-constrained hardware.
- **Interface Separation:** Modular Layer Decoupling: The parser, projection, terrain analysis, and pond selection layers are decoupled behind pure data-structure interfaces (NumPy arrays, dataclasses, Shapely shapes), allowing any layer to be swapped or integrated into a CLI or batch worker.

C. Documentation Quality :-

Complete documentation is maintained across the repository: README.md provides quick-start guides and deployment instructions; IMPLEMENTATION_PLAN.md documents the architectural decisions and constraints; and FastAPI automatically serves live OpenAPI/Swagger documentation at /docs.

D. Catchment Identification and Estimation Rigor :-

Hydrological calculations are mathematically validated. The Priority-Flood algorithm eliminates artificial depressions, the D8 flow model accurately distributes drainage, and upstream recursive tracing ensures the delineated catchment area matches the cell count product (541 cells * 100 m2 = 54,100 m2). All polygon boundaries are guaranteed to be closed, topologically valid rings.

6. Automated Testing and Verification :-

The codebase is covered by an automated test suite of 40 tests across unit and integration layers:

- **tests/test_parser.py:** 12 tests covering namespace-agnostic parsing, XML syntax errors, coordinate validation, and Placemark filtering.
- **tests/test_terrain.py:** 15 tests covering Priority-Flood fill on synthetic pits, bowls, flat planes, and staircases, vector D8 direction, accumulation conservation, and slope calculation.
- **tests/test_integration.py:** 13 tests exercising the FastAPI TestClient end-to-end against real KML files, verifying schema compliance, numerical sanity, and 400 error codes.

Test Execution Result: All 40 tests pass with zero failures on both local development machines and the remote deployment box.

7. AI Tools Citation :-

I have used Chatgpt for help in some areas.